Waiting Time Prediction in Queue Management: Leveraging Machine Learning Approach

Tapodhir Karmakar Taton
Electrical and Electronic Engineering
University of Rajshahi, Bangladesh
tapodhirtaton@gmail.com

Bipin Saha
Electrical and Electronic Engineering
University of Rajshahi, Bangladesh
bipinsaha.bd@gmail.com

Amena Akter
Electrical and Electronic Engineering
University of Rajshahi, Bangladesh
amena.akhi5@gmail.com

Md. Johirul Islam*
Physics
Rajshahi University of Engineering & Technology, Bangladesh
johirul@phy.ruet.ac.bd

Shaikh Khaled Mostaque*
Electrical and Electronic Engineering
University of Rajshahi, Bangladesh
misha@ru.ac.bd

***Abstract*—This research highlights a novel approach to queue management systems (QMS), offering a scalable, cost-effective, and intelligent solution to contemporary challenges. The proposed study evaluates regression algorithms along with Long Short Term Memory (LSTM) model to enable real-time tracking and estimation, considering the continuous nature of the prediction of the waiting time required by a client before taking a service. Among the models evaluated for waiting time prediction, the Voting Regressor(VR) demonstrates superior performance across simulation scenarios, achieving the highest R^2 score of 0.944 and the lowest mean absolute error (MAE) of 1.828 minutes. The QMS dynamically allocates customer requests to available service counters, optimizing resource utilization and improving operational efficiency. This work addresses the limitations of traditional physical systems, such as the need for physical presence, inefficient space and time utilization, and limited scalability.**

***Index Terms*—Queue Management, Machine Learning, Waiting Time Prediction**

I. Introduction

Queue Management Systems (QMS) play a crucial role in enhancing customer service experiences by providing real-time information on waiting status, estimated waiting times, and service counter assignments. The COVID-19 pandemic has introduced additional challenges, necessitating the enforcement of social distancing and the avoidance of mass gatherings, particularly in high-traffic areas such as banks, ticket counters, retail shops, telecommunication companies, and other service sectors. The implementation of QMS in such environments provides an effective solution for crowd management ensuring compliance with social distancing guidelines.

Over the years, numerous studies have explored various strategies for customer congestion control, including Shortest Processed First (SPF), First Come First Serve (FCFS), Single Queue (SQ), Multiple Queue (MQ), Diffuse Queue (DQ), and Head of Queue (HQ) [1]. Among these, FCFS is widely recognized as the fairest approach, ensuring that each customer is treated equally. Previous implementations of FCFS-based electronic queue management systems, utilizing PIC microcontrollers, have demonstrated the capability to manage up to 100 consumers across multiple service counters [2]. These systems typically involve displaying token and service counter numbers on either a general display or independent panels for each service desk, requiring clients to obtain a token before receiving service.

Advancements in QMS technology have included the use of dual PIC microcontrollers in master-slave configurations, enabling communication between operators and clients via multiple 7-segment displays [3]. Some systems have incorporated voice announcements using speakers, which, while enhancing customer awareness, increase system costs and power consumption [4]. Other approaches have utilized GSM transceivers to facilitate communication between subscribers and server computers via SMS, necessitating precise synchronization [5], [6]. Sensor-based methods, such as those employing infrared (IR) sensors to track customer flow, have also been explored, achieving accuracy levels of 80% and 90% for counting and real-time data representation respectively [7]. None of the existing systems incorporate waiting time prediction features. Traditional physical queue management systems (QMS) face considerable limitations, such as the necessity for physical presence, inefficient utilization of space and time, high operational costs, and limited scalability [8]. Carvalho *et al.* [20] approached this issue using machine learning to predict time overflow in bank teller queues, focusing on the binary classification problem of determining whether a customer's waiting time exceeds 30 minutes. While classification algorithms effectively predict discrete outcomes by mapping input variables to distinct categories [10], they are less suited for predicting continuous variables such as waiting time.

The primary contribution of this paper lies in developing a software-driven QMS utilizing Python and a REST API framework based on Flask. The proposed system, based on the FCFS method, integrates a machine learning technique for waiting time prediction illustrated in the following sections. Since predicting waiting time involves numerical forecasting, regression analysis is utilized as the most appropriate approach

for this task [10]. The system dynamically allocates requests to available service counters, offering live tracking and real-time waiting time estimation.

II. Waiting Time Prediction

The study predicts client waiting times from complex datasets employing machine learning [9]. As depicted in Figure 1, the research process commences with the generation of the dataset. Following this, several steps of data preprocessing and feature selection have been performed. The refined dataset is then fed into some selected models. These models have been trained using the dataset, and their performances have been subsequently evaluated using various metrics.

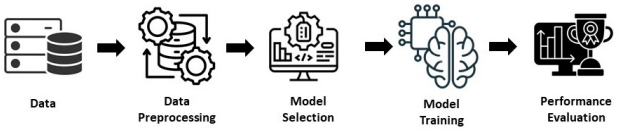


Fig. 1: Flow diagram of waiting time prediction process

A. Dataset

The dataset used in this study simulates three distinct pairs of arrival rates and mean service times, where each row represents one month of data and contains at least 6,500 data points. Each data point includes seven features along with a target variable labeled "waiting_time." The data has been generated using varying arrival rates (0.5681/min, 0.4824/min, 0.3967/min) and mean service times (5.0233 min, 6.0177 min, 7.0121 min). These values are selected based on the properties of a real bank dataset [20], ensuring they fall within the dataset's standard deviation and the second variance point, covering 95% of data. Moreover, addition of Additive White Gaussian Noise (AWGN) replicating real-world circumstances, improves the resilience of model assessment in noisy situations. The data generation process is illustrated with Algorithm 1 using Pseudocode, and a detailed description of the dataset is provided in Table I.

TABLE I: Dataset Description

Feature Name	Description
date	Date of customer arrival
arrival_time	Joining time in the queue
start_time	Service start time
end_time	Service end time
waiting_time	The amount of time clients had to wait to receive service
service_time	The amount time taken for service
counter	Unique number of service provider counter

B. Data Preprocessing

Data preprocessing techniques have been applied to prepare raw data for analysis. Timestamp features are converted to minutes for a balanced level of detail and simplicity [12]. A secondary feature, 'queue_length,' has been produced here to represent the number of clients waiting for service when a new client arrives. To calculate this, each client's data is filtered according to date, and entries are identified when the service start time occurs after the client's arrival time. The counts of such entries have been assigned as the queue length for that client. This procedure is then applied to all clients in the dataset, as shown in Figure 2.

Algorithm 1 Customer Arrival and Service Simulation with Varying Rates

Arrival rates $\{0.5681, 0.4824, 0.3967\}$, Mean service times $\{5.0233, 6.0177, 7.0121\}$ Customer log with arrival, waiting, and service times

Initialization:
Counters, Start time, Simulation duration, Date range
Empty lists for customer_log, counters, time, total_time
Waiting_time = 0, Customer_index $i = 1$

Daily simulation:
for each day **do** Select arrival_rate and mean_service_time

for each minute of the day **do**
if customer arrives (based on arrival_rate) **then**
if counter available **then** Assign customer to counter
Set waiting_time = 0

else Compute waiting_time and assign to the first available counter

Generate service time:
service_time = **get_service_time(mean_service_time)**
Compute start and end times
Log customer details: (arrival_time, start_time, end_time, waiting_time, service_time, counter)

Noise Implantation:
Define parameters for Additive White Gaussian Noise (AWGN): noise_percentage, time_noise_std_dev, service_noise_std_dev
Randomly select rows in customer_log

for each selected row **do** Apply AWGN to 'arrival_time', 'waiting_time', and 'service_time'

Post-simulation:
Convert customer_log to DataFrame and save to Excel
return customer_log

C. Model Selection

Since this is a regression problem, ensemble methods are employed due to their demonstrated superiority over single predictors in various tasks [13]. Random Forest (RF), which combines bagging and random subspace techniques, is effective in avoiding over-fitting and excels in both classification and regression [14], [15]. Extra Trees (ET) extends RF by being computationally efficient and highly randomized

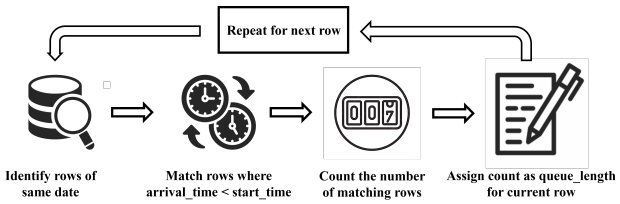


Fig. 2: Calculation of the 'queue_length' feature

[16], [17]. While the Gradient Boosting Regressor (GBR) sequentially builds trees to minimize loss functions, reducing variance and bias through forward stage-wise modeling. The Hist Gradient Boosting Regressor (HGBR), a histogram-based method, offers faster training and lower memory usage [18]. Finally, the Voting Regressor (VR) aggregates predictions from multiple models, trained on the full dataset, to enhance forecast accuracy [14], [19]. Being effective with the sequential data, LSTM model has also been evaluated alongside the regression models. LSTM is particularly suited for time-series data due to its ability to capture long-term dependencies [21], making it an ideal choice for this application where client arrival rates and service times exhibit temporal dependencies.

III. Implementation of QMS

The proposed QMS integrates the best-performing model to provide clients with tentative waiting times upon joining a queue, based on three different simulated datasets resulting from the implementation of Algorithm 1 described in section II(A). The backend includes a dedicated server to run the predictive model, ensuring real-time data is processed quickly. As a client joins the queue, the system refreshes the data and feeds it to the model to display the estimated waiting time. This architecture ensures efficient interaction between the QMS and the predictive model, delivering prompt and accurate predictions.

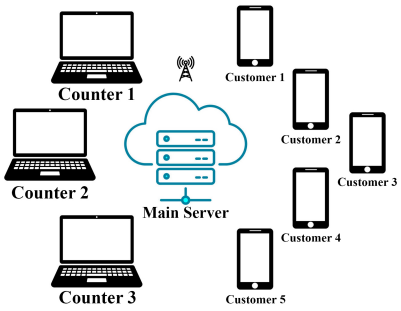
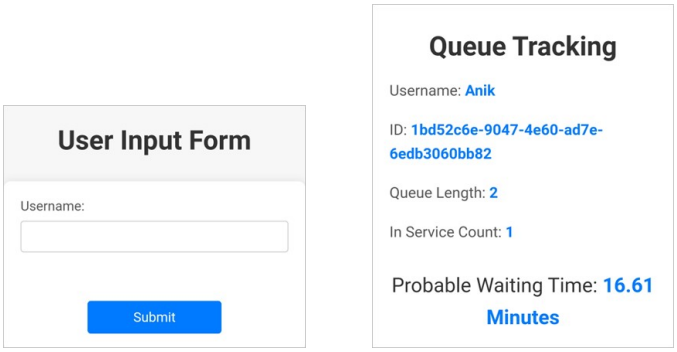


Fig. 3: Proposed Model of QMS

A. User Input and Tracking

In this study, we consider clients to use personal devices for submission of a service request via a local server. After submission of the required information, clients can view real-time queue details and their position on a tracking page. The tracking page is dynamically updated by the pre-trained model on the backend along with forecasting waiting times based on user and queue data.



(a) User Input Form (b) Queue Tracking Page

Fig. 4: User Input & Tracking

B. Counter Initialization

As illustrated in Figure 6, the QMS automatically assigns clients to service counters based on the FCFS principle, ensuring fair and orderly service [1]. Each counter is uniquely numbered for easy management. When a client's service is completed, the service provider clicks "Next", prompting the system to assign the next client. By clicking the "Close Counter" button, a service provider stops new client assignments to the counter, while the current client is served until completion. This automated process ensures continuous service flow and maximizes counter utilization. The QMS is also scalable, allowing easy addition of new counters as demand grows without disrupting operations.

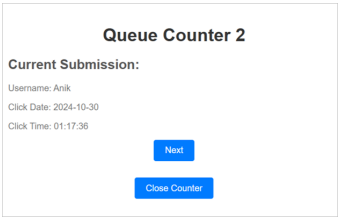


Fig. 5: Service Counter

C. Administration

The administration page of the proposed QMS ensures efficient and transparent service delivery. It automatically refreshes to display the most current queue status. When a client enters their information, they are added to the queue with a "not taken" status, indicating they are awaiting assignment to a service counter. Once assigned, their status updates to "in service," along with the assigned counter number. After service completion, the status changes to "completed." This seamless transition from "not taken" to "in service" and finally "completed" ensures orderly queue management. The automatic refresh feature keeps information accurate and up-to-date, minimizing the need for manual updates and enhancing overall system efficiency.

TABLE II: Performance metrics for different models based on arrival rates and mean service times

Dataset Parameters	Models	R^2 Score	MAE	MSE	RMSE
Scenario I: Arrival Rate = 0.4824/min Mean Service Time = 6.0177 min	Random Forest Regressor	0.93	1.947	7.82	2.796
	Extra Trees Regressor	0.92	1.973	8.618	2.935
	Gradient Boosting Regressor	0.922	2.205	8.828	2.971
	Hist Gradient Boosting Regressor	0.936	1.967	7.189	2.681
	Voting Regressor	0.944	1.828	6.314	2.512
	LSTM (128)	0.912	2.398	10.286	3.207
Scenario II: Arrival Rate = 0.5681/min Mean Service Time = 5.0233 min	Random Forest Regressor	0.92	1.921	7.628	2.762
	Extra Trees Regressor	0.909	1.983	8.719	2.952
	Gradient Boosting Regressor	0.893	2.327	10.294	3.208
	Hist Gradient Boosting Regressor	0.919	2.027	7.736	2.781
	Voting Regressor	0.93	1.853	6.704	2.589
	LSTM (128)	0.911	2.116	8.309	2.882
Scenario III: Arrival Rate = 0.3967/min Mean Service Time = 7.0121 min	Random Forest Regressor	0.919	2.194	9.803	3.131
	Extra Trees Regressor	0.906	2.252	11.387	3.374
	Gradient Boosting Regressor	0.90	2.531	12.009	3.465
	Hist Gradient Boosting Regressor	0.92	2.249	9.576	3.094
	Voting Regressor	0.936	2.054	7.744	2.782
	LSTM (128)	0.90	2.45	11.042	3.323

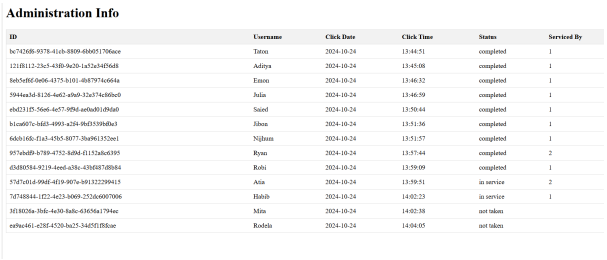


Fig. 6: Administration Page

IV. Result

In this section, the overall performance of different models is evaluated based on performance metrics in three different scenarios. We evaluate the effectiveness of multiple regression models with the LSTM model to determine the most accurate approach for predicting waiting times.

Table II illustrates the performance evaluation of five regression models—RF, ET, GBR, HGBR, and VR and a Recurrent Neural Network model—LSTM across three distinct scenarios, where arrival rates and mean service times are paired randomly to avoid selection bias. In the first scenario, where the arrival rate is 0.4824/min and the mean service time is 6.0177 min, the VR achieves the highest R^2 score of 0.944, indicating a strong explanatory power. In addition to that, the lowest Mean Absolute Error (MAE) of 1.828 and Root Mean Squared Error (RMSE) of 2.512 reflect minimal prediction error. This trend persists in the second scenario (arrival rate = 0.5681/min, mean service time = 5.0233 min), where the VR again secures an R^2 of 0.93, further establishing its reliability. In the third scenario (arrival rate = 0.3967/min, mean service time = 7.0121 min), the VR maintains an R^2 of 0.936, demonstrating consistent performance across varying conditions. In contrast, less accurate predictions are shown by the GBR and ET Regressors, which have larger MAE and RMSE values and relatively lower R^2 scores. The LSTM model, configured with a sequence length of 128, demonstrates performance that is closely comparable to the other models across all three scenarios. These results underscore the VR's superiority in capturing complex relationships in the dataset and suggest its robustness for applications necessitating accurate predictions based on the dynamics of arrival rates and service times.

Conclusion

This study presents a comprehensive web-based Queue Management System that integrates machine learning to predict waiting times. By generating a simulated dataset of queueing scenarios and training it on some ensemble tree and bagging boosting models, we identify the VR to be the most accurate model. The outperforming model is then implemented in the QMS backend developed using Python and a Flask-based REST API. This system accepts user details as input and invokes the predictive model to generate waiting time estimates based on both user-specific and queue-related data. This system not only predicts waiting times but also offers real-time queue tracking. Thus, incorporating machine learning into a web-based QMS can provide a practical, scalable, and cost-effective solution for enhancing the efficiency of queue management in various scenarios. Future works may explore incorporation of additional features and load balancing mechanisms while accounting for high traffic or other demanding scenarios.

References

[1] Uddin, M., Rashid, M., Mostafa, M. & Ahmed, S. Automated queue management system. *Glob. J. Manag. Bus. Res. An Adm. Manag*. **16**, 1-9 (2016)

[2] Hossain, M., Hossain, M., Hossen, M. & Rahman, M. Design and development of microcontroller based electronic queue control systems. *IEEE Technology Students' Symposium*. pp. 48-52 (2011), https://api.semanticscholar.org/CorpusID:37336096

[3] Roy, B. & Venugopal, A. Novel Intelligent System for Efficient Queue Management. *ISSN International Journal Of Advanced Research In Electrical Electronics And Instrumentation Engineering*. **2**, 2320-3765 (2013)

[4] Zadidul, A. & Mahbub, M. Low power microcontroller based intelligent token number speaker and display system. *International Journal Of Engineering And Technology*. **3**, 199 (2011)

[5] Tahat, A. Implementation of an SMS-based telemedicine system for patient electrocardiogram monitoring. *Proceedings Of The 4th IASTED International Conference On Telehealth And Assistive Technologies (AT'08)*. pp. 223-228 (2008)

[6] Arun, R. & Priyesh, P. Smart queue management system using gsm technology. *Advanced In Electronic And Electric Engineering*. **3**, 941-950 (2013)

[7] Gimba, U., Okoronkwo, C., Yusuf, M., Musa, A. & Ali, M. Queue monitoring system for bank. *Dutse Journal Of Pure And Applied Sciences (DUJOPAS)*. **6**, 269-276 (2020)

[8] Mallari, M., Guintu, J., Magalong, Y. & Yap, D. CLIQUE: A Web-Based Queue Management System with Real-Time Queue Tracking and Notification of Units for Angeles University Foundation Office of the University Registrar.

[9] Joseph, J., Senith, S., Kirubaraj, A. & Ramson, J. Machine Learning for Prediction of Wait Times in Outpatient Clinic. *Procedia Computer Science*. **215** pp. 230-239 (2022)

[10] Kinaneva, D., Hristov, G., Kyuchukov, P., Georgiev, G., Zahariev, P. & Daskalov, R. Machine learning algorithms for regression analysis and predictions of numerical data. *2021 3rd International Congress On Human-Computer Interaction, Optimization And Robotic Applications (HORA)*. pp. 1-6 (2021)

[11] Obaid, H., Dheyab, S. & Sabry, S. The impact of data pre-processing techniques and dimensionality reduction on the accuracy of machine learning. *2019 9th Annual Information Technology, Electromechanical Engineering And Microelectronics Conference (iemecon)*. pp. 279-283 (2019)

[12] Weld, D. & De Kleer, J. Readings in qualitative reasoning about physical systems. (Morgan Kaufmann,2013)

[13] Brown, G., Wyatt, J., Harris, R. & Yao, X. Diversity creation methods: a survey and categorisation. *Information Fusion*. **6**, 5-20 (2005)

[14] Ren, Y., Zhang, L. & Suganthan, P. Ensemble classification and regression-recent developments, applications and future directions. *IEEE Computational Intelligence Magazine*. **11**, 41-53 (2016)

[15] Breiman, L. Random forests. *Machine Learning*. **45** pp. 5-32 (2001)

[16] John, V., Liu, Z., Guo, C., Mita, S. & Kidono, K. Real-time lane estimation using deep features and extra trees regression. *Image And Video Technology: 7th Pacific-Rim Symposium, PSIVT 2015, Auckland, New Zealand, November 25-27, 2015, Revised Selected Papers 7*. pp. 721-733 (2016)

[17] Saha, B., Islam, M., Dipto, A. & Mostaque, S. An Efficient Approach for Appearance Based Eye Gaze Estimation with 13 Directional Points. *2021 International Conference On Computer, Communication, Chemical, Materials And Electronic Engineering (IC4ME2)*. pp. 1-5 (2021)

[18] Elith, J., Leathwick, J. & Hastie, T. A working guide to boosted regression trees. *Journal Of Animal Ecology*. **77**, 802-813 (2008)

[19] Ko, J. & Byun, Y. Analyzing factors affecting micro-mobility and predicting micro-mobility demand using ensemble voting regressor. *Electronics*. **12**, 4410 (2023)

[20] Mourõo, R., Carvalho, R., Carvalho, R. & Ramos, G. Predicting waiting time overflow on bank teller queues. *2017 16th IEEE International Conference On Machine Learning And Applications (ICMLA)*. pp. 842-847 (2017)

[21] Lindemann, B., Müller, T., Vietz, H., Jazdi, N. & Weyrich, M. A survey on long short-term memory networks for time series prediction. *Procedia Cirp*. **99** pp. 650-655 (2021)

[22] Sherstinsky, A. Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D: Nonlinear Phenomena*. **404** pp. 132306 (2020)